

ĐỀ KIỂM TRA: MICRO SERVICES VỚI SPRING BOOT – QUẢN LÝ SINH VIÊN & KHÓA HỌC

Môn học: Microservices và Spring Boot

Thời gian làm bài: 180 phút

Công nghệ: Java 17+, Spring Boot 3.5.14, Spring Cloud, Maven/Gradle, Docker/Kafka (Optional), PostgreSQL

I. MỤC TIÊU BÀI KIỂM TRA

Đánh giá khả năng xây dựng hệ thống Microservices cơ bản, bao gồm: Quản lý dịch vụ qua Service Registry, điều phối yêu cầu qua Gateway và giao tiếp giữa các dịch vụ.

II. YÊU CẦU HỆ THỐNG

Học viên phải xây dựng hệ thống gồm 4 thành phần:

- Service Registry (Eureka Server):** Quản lý danh bạ dịch vụ.
- API Gateway:** Điều phối và định tuyến request.
- Student Service:** Quản lý thông tin sinh viên.
 - Bảng `students` thuộc database `student_db`:
 - `id` (PK, Long)
 - `name` (String)
 - `email` (String)
 - `enrolled_courses` (Array of Long – IDs khóa học)
- Course Service:** Quản lý khóa học.
 - Bảng `courses` thuộc database `course_db`:
 - `id` (PK, Long)
 - `course_name` (String)
 - `capacity` (Integer) – Số lượng học viên tối đa

- `credits` (Integer)
-

III. CÁC NHIỆM VỤ CỤ THỂ

Task 1: Hạ tầng (30 điểm)

- Khởi tạo Eureka Server.
 - Cấu hình Student Service và Course Service tự động đăng ký.
 - Cấu hình API Gateway:
 - Route `/api/students/**` → Student Service
 - Route `/api/courses/**` → Course Service
 - Viết Global Filter log tất cả request (Time, Method, URI).
-

Task 2: Nghiệp vụ & Giao tiếp (50 điểm)

- **Student Service:** API POST `/api/students/enroll` nhận payload `{studentId, courseId}` để đăng ký khóa học.
 - **Course Service:** API GET `/api/courses/{courseId}` trả về thông tin khóa học.
 - **Luồng giao tiếp:** Khi gọi `/api/students/enroll`, Student Service phải kiểm tra:
 - Khóa học còn chỗ không (`capacity > 0`) qua Course Service.
 - Cách giao tiếp:
 - **Cách A:** Dùng OpenFeign (đồng bộ)
 - **Cách B:** Dùng Kafka (bất đồng bộ, Student gửi event, Course lắng nghe và phản hồi).
 - Cập nhật số lượng học viên và danh sách `enrolled_courses` tương ứng.
-

Task 3: Phân tích kiến trúc (20 điểm – README.md)

1. Giải thích cơ chế **Service Discovery**: tại sao Gateway không gọi trực tiếp IP/Port?
 2. Nếu số lượng request tăng đột biến, làm thế nào để scale Student Service mà không thay đổi cấu hình Gateway?
 3. So sánh ưu/nhược điểm **OpenFeign (Đồng bộ)** vs **Kafka (Bất đồng bộ)** trong bài toán này.
-

IV. HƯỚNG DẪN NỘP BÀI

- Cấu trúc thư mục: Nộp link repository GitHub.
- Yêu cầu kỹ thuật: Các service phải chạy độc lập.
- File README.md:
 - Chứa sơ đồ kiến trúc (text/ảnh)
 - Trả lời Task 3.

IV. HƯỚNG DẪN NỘP BÀI

- **Cấu trúc thư mục:** Nộp link Repository trên GitHub.
- **Yêu cầu kỹ thuật:** Các service phải chạy được độc lập.
- **File đính kèm:** Phải có file `README . md` chứa sơ đồ kiến trúc (dạng text hoặc hình ảnh) và câu trả lời Task 3.

V. THANG ĐIỂM CHI TIẾT

Mục tiêu	Tiêu chí đánh giá	Điểm
Hạ tầng	Eureka Server nhận diện đầy đủ 3 service	15
Gateway	Định tuyến đúng + Filter hoạt động chính xác	20
Kết nối	Giao tiếp giữa Student Service và Course thành công	15
Nghiệp vụ	Xử lý logic đúng theo yêu cầu (Đăng kí môn, trừ số lượng)	20
Thiết kế các API	Hoạt động tốt với các case kiểm thử	15
Trả lời câu hỏi	Nắm được các kiến thức và hiểu kiến trúc	15
TỔNG		100